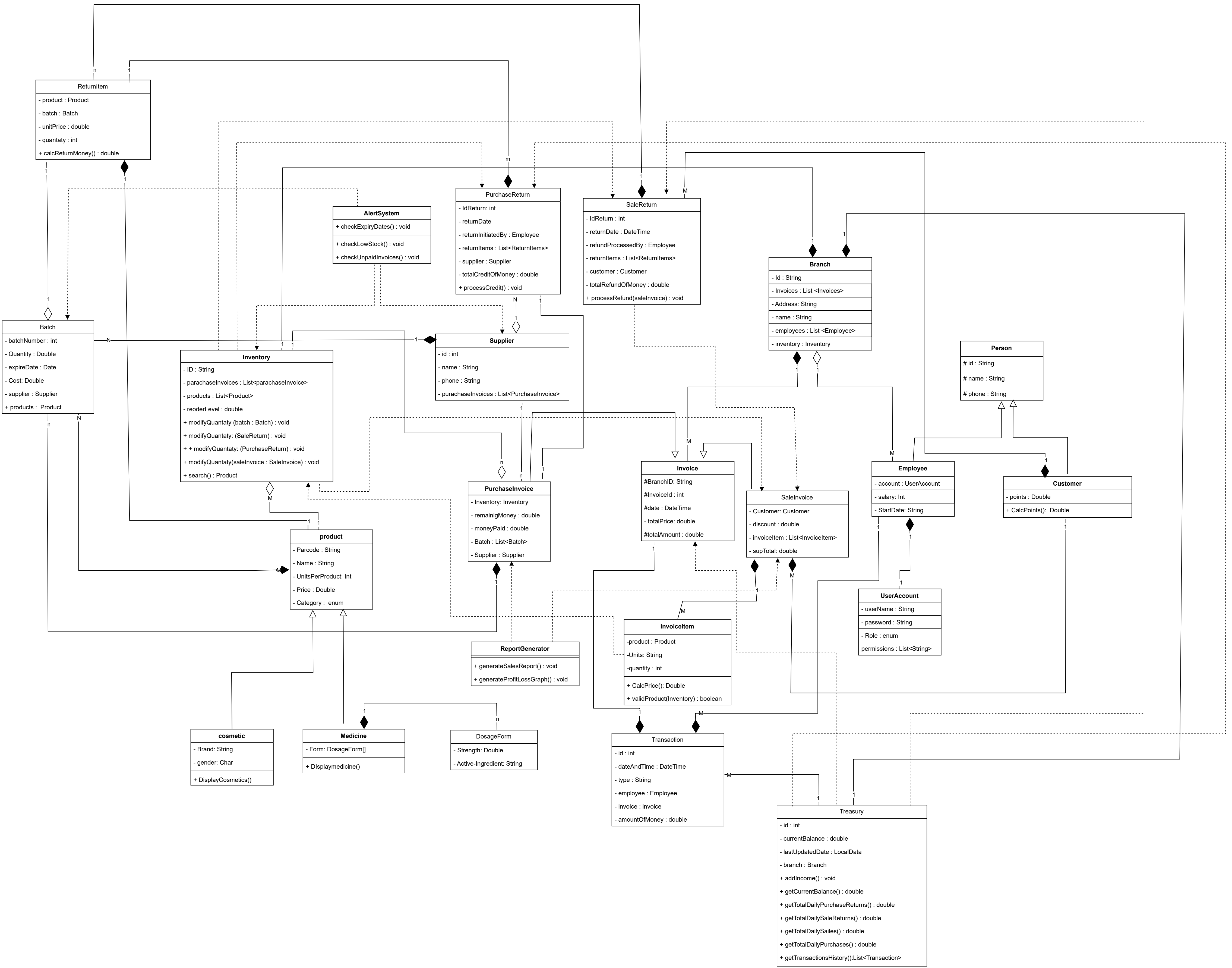




System Design Description

Overview

The Pharmacy System is designed to automate and manage the operations of a pharmacy, including the sale and purchase of medicines and cosmetics, inventory control, financial tracking, and report generation. The system maintains detailed records of customers, employees, suppliers, invoices, and transactions to ensure efficient workflow and accurate data management. The UML class diagram provides a structural representation of the system's main entities and their relationships, illustrating inheritance, composition, and association across the different modules.

Core Entity Classes

Product Hierarchy

At the core of the system lies the **Product** class, which represents any item available for sale in the pharmacy. It includes essential attributes such as parcode, name, unitsPerProduct, price, and category, defining the basic identity of a product.

Two specialized subclasses extend this base class: Medicine and Cosmetic.

The **Medicine** class introduces the pharmaceutical aspects of the product through a composition relationship with the DosageForm class, which specifies details such as strength and active ingredient. The method DisplayMedicine() is used to present medicine-related data to the user.

The **Cosmetic** class extends Product with attributes such as brand and gender, and includes the method DisplayCosmetics() to display cosmetic-specific information.

This inheritance structure allows the system to manage both product types efficiently while maintaining shared characteristics within the parent Product class.

Inventory Management

The **Inventory** class is a critical component that manages the stock levels, products, and purchase invoices within each branch. It contains lists of Product and PurchaseInvoice objects and uses multiple overloaded modifyQuantity() methods to adjust item quantities following different transactions such as sales, returns, and purchases.

The `search()` method allows retrieval of a specific product, while the `reorderLevel` attribute ensures that products are restocked before reaching critical levels. The inventory is directly linked to each branch, allowing localized yet synchronized management of stock data.

Human Entities

Person Hierarchy

The **Person** class serves as a generalization for all individuals who interact with the system. It includes basic identifying attributes such as `id`, `name`, and `phone`.

Two main subclasses extend this class: **Customer** and **Employee**.

The **Customer** class introduces a `points` attribute for the loyalty system, along with the method `CalcPoints()` to compute reward points based on purchase amounts.

The **Employee** class extends **Person** with employment-specific information including `salary`, `startDate`, and a `UserAccount` object that holds authentication data (`username`, `password`, `role`, and `permissions`). This composition establishes a secure access control mechanism based on employee roles.

User Account

The **UserAccount** class defines the system's authentication and authorization layer. It ensures that each employee is associated with a distinct user identity and set of permissions. This design supports role-based access control (RBAC), improving data security and operational accountability.

Branch and Operational Classes

Branch

The **Branch** class represents each physical pharmacy location. It contains the `id`, `name`, `address`, and maintains associations with employees, invoices, and an inventory instance. This encapsulation allows each branch to operate autonomously while adhering to the same global system structure.

Invoice Hierarchy

The **Invoice** class acts as a superclass that generalizes all billing activities in the system. It defines attributes such as `branchId`, `invoiceId`, `date`, `totalPrice`, and `totalAmount`.

Two subclasses extend its functionality:

The **PurchaseInvoice** class represents transactions where the pharmacy acquires products from suppliers. It maintains details such as inventory, moneyPaid, remainingMoney, batches, and the associated supplier.

The **SaleInvoice** class represents customer sales and includes customer, discount, invoiceItems, and subTotal. Each sale consists of multiple InvoiceItem objects that specify the product, units, and quantity, and implement CalcPrice() and validProduct() methods for price computation and stock validation.

Batch and Supplier

The **Batch** class handles detailed information for each lot of a product, including batchNumber, quantity, expireDate, cost, and supplier. This design enables precise monitoring of product expiration and cost management.

The **Supplier** class stores supplier-related information such as id, name, phone, and a list of associated PurchaseInvoices, enabling traceability and accountability in procurement activities.

Returns and Refund Management

The system supports both sales and purchase returns through two dedicated classes: SaleReturn and PurchaseReturn.

The **SaleReturn** class records customer refund operations, containing attributes such as idReturn, returnDate, refundProcessedBy, returnItems, customer, and totalRefundOfMoney. The processRefund() method executes refund operations and updates financial records.

The **PurchaseReturn** class manages supplier return transactions, including returnDate, returnInitiatedBy, returnItems, supplier, and totalCreditOfMoney, using the processCredit() method to process supplier credit adjustments.

Both classes depend on the ReturnItem class, which defines the returned product, batch, unitPrice, and quantity, and includes the calcReturnMoney() method to compute refund amounts.

Financial Management

The **Treasury** class manages financial resources at the branch level. It maintains attributes such as id, currentBalance, lastUpdatedDate, and the related branch. The class provides several analytical and transactional methods including addIncome(), getCurrentBalance(), getTotalDailySales(), getTotalDailyPurchases(), getTotalDailySaleReturns(), and

getTransactionsHistory().

Each financial operation is represented by a **Transaction** object that captures the dateAndTime, type, employee, invoice, and amountOfMoney. This structure ensures accurate financial tracking and accountability across all monetary activities.

System Monitoring and Reporting

To enhance operational control, the system incorporates two monitoring and analytics classes: AlertSystem and ReportGenerator.

The **AlertSystem** automatically identifies and reports critical situations such as product expiry, low stock levels, or unpaid invoices using methods like checkExpiryDates(), checkLowStock(), and checkUnpaidInvoices().

The **ReportGenerator** class supports decision-making by generating analytical reports and visual graphs through methods generateSalesReport() and generateProfitLossGraph(), offering insights into business performance and profitability trends.

Relationships and Design Principles

The UML model follows object-oriented design principles emphasizing reusability, modularity, and encapsulation. **Inheritance** is utilized to generalize entities such as *Product*, *Person*, and *Invoice*, while **composition** is applied in relationships like *Branch–Inventory* and *Invoice–InvoiceItem*, ensuring strong data integrity and dependency control. **Aggregation** is also represented in several relationships, such as between *Branch* and *Employees*, or *Inventory* and *Products*, where one class groups or manages multiple instances of another without directly owning their lifecycle. In addition, **dependency** relationships appear between operational components—for example, the *AlertSystem* relies on *Inventory* to check expiry dates and stock levels, while the *ReportGenerator* depends on data from *Treasury* and *Invoice* to produce analytical reports. **Associations** between entities—such as between *Supplier* and *PurchaseInvoice* or *Customer* and *SaleInvoice*—reflect real-world business interactions, reinforcing the logical flow of pharmacy operations.

Overall, the UML class diagram represents a cohesive, scalable, and maintainable architecture that supports all essential business processes within a modern pharmacy environment.